MétéoSentinel dossier technique

Validation 1 · Architecture web distribuée · 2026–2027

Cette conception propose des activités et des créneaux expliqués à partir de météo, qualité de l’air et préférences. Le dossier décrit responsabilités, modèles, moteurs, API, échanges, sécurité et organisation. Le métier sera développé après validation de conception.

Six étudiants possèdent chacun un service métier. DataHub constitue le service commun de traçabilité, de qualité et d’analyse des données.

Membre	Service	Moteur
Nour Maroufi	Weather	PostgreSQL
Dhia Borzli	Forecast	MongoDB
Mohamed Hedi Bejaoui	Alert	Redis
Kaouther Nalouti	Air Quality	MySQL
Ahmed Ben Mahmoud	User	MariaDB
Mohamed Aziz Sayhi	Activity	Qdrant (vectoriel)
Toute l’équipe	DataHub	ClickHouse

Conception S4 : architecture, modèles, contrats et organisation du développement.

Sujet acteurs et fonctionnalités

MétéoSentinel aide un utilisateur à choisir une activité et un créneau à l'échelle d'une ville en croisant météo, prévisions, qualité de l'air et préférences. Un sportif, une association ou un étudiant compare des créneaux expliqués. Le système ne remplace pas une vigilance officielle et ne calcule pas un itinéraire.

User stories et critères de recette

US-1 – Visiteur

Je souhaite consulter la météo et l'AQI d'une ville.

Recette : la source, l'heure et l'échelle AQI sont visibles ; indisponible ne vaut pas zéro.

US-2 – Utilisateur

Je souhaite chercher une activité selon mes centres d'intérêt.

Recette : la similarité sémantique a un modèle/version documentés et les raisons sont affichées.

US-3 – Utilisateur

Je souhaite comparer des créneaux de 1 à 4 heures.

Recette : aucune fenêtre ne traverse une heure manquante ; seuils météo explicités.

US-4 – Utilisateur

Je souhaite gérer préférences et consentement.

Recette : le serveur lie /me à mon identité ; le retrait cesse les futures notifications.

US-5 – Utilisateur

Je souhaite comprendre une alerte.

Recette : classe, facteurs dominants, version de règle et date des données sont visibles.

US-6 – Administrateur

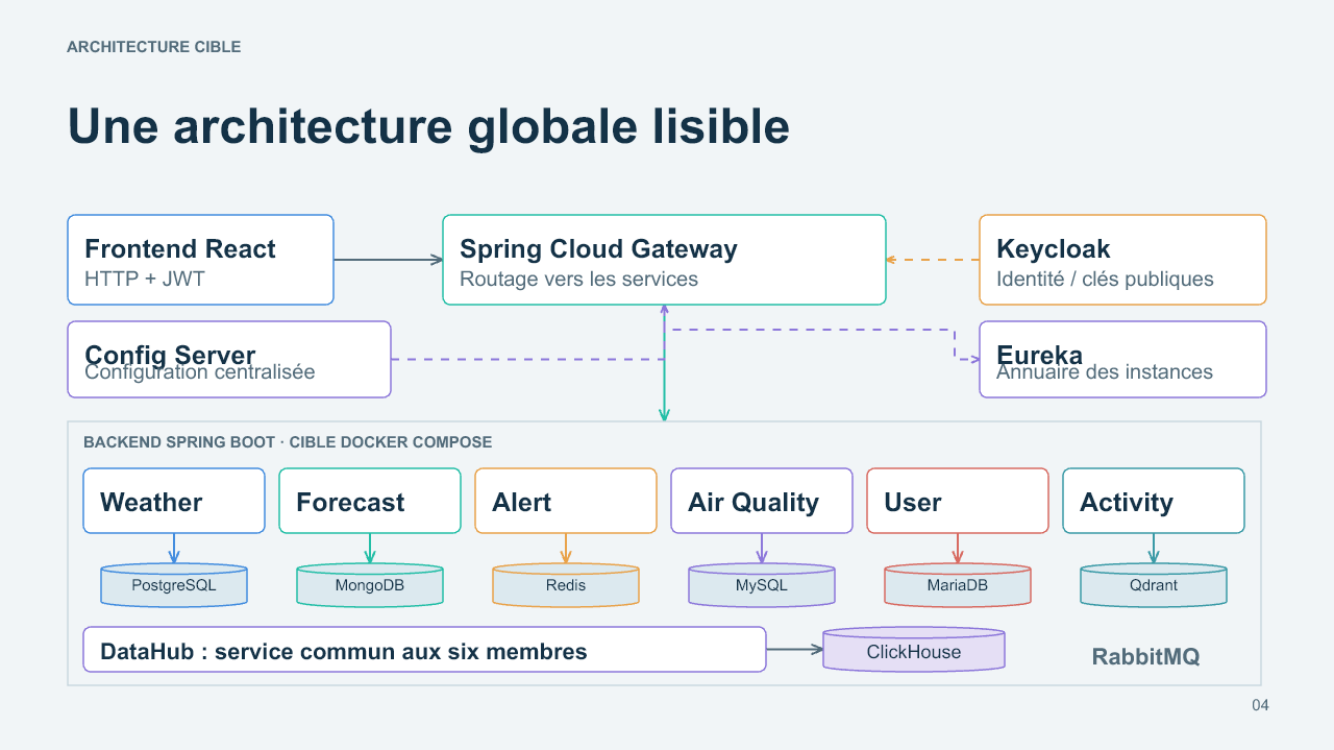
Je souhaite contrôler qualité et erreurs des prévisions.

Recette : les références sont typées observation/réanalyse et les métriques séparées par horizon.

Périmètre réaliste

MVP après validation : trois villes tunisiennes, météo actuelle, 48 heures de prévision, un indice AQI documenté, catalogue limité, trois activités et alertes dans l'application. Hors périmètre initial : SMS, routage GPS, entraînement de modèle ML, promesse sanitaire. La recherche vectorielle utilise un modèle existant figé ; l'évaluation du modèle et son ingestion restent à développer.

Architecture et infrastructure



Frontend React > Spring Cloud Gateway > sept services métier Spring Boot. Eureka est l'annuaire des instances. Config Server fournit configuration centralisée, sans secrets dans Git. Keycloak gère OIDC/PKCE et émet les JWT ; Gateway et services valident signature, émetteur, expiration, audience puis les autorisations du métier. Keycloak ne fait pas transiter chaque requête métier. Les bases sont exclusives, une par service, avec credentials et réseaux isolés.

Spring Boot 3.5.16 / Java 17 / Spring Cloud 2025.0.1. Ces versions Boot/Cloud appartiennent à la même ligne compatible : https://spring.io/projects/spring-cloud/ . OpenFeign pour les contrats REST ; RabbitMQ pour les événements ; Docker Compose pour l'environnement cible.

Services, bases et responsables

- Weather : Nour Maroufi – **PostgreSQL**, weather_db. - Forecast : Dhia Borzli – **MongoDB**, forecast_db. - Alert : Mohamed Hedi Bejaoui – **Redis**, alert_db. - Air Quality : Kaouther Nalouti – **MySQL**, air_quality_db. - User : Ahmed Ben Mahmoud – **MariaDB**, user_db. - Activity : Mohamed Aziz Sayhi – **Qdrant (vectoriel)**, activity_db. - DataHub : Toute l'équipe – **ClickHouse**, datahub_db.

Technologies et périmètre S4

Technologie avancée

Activity utilise la recherche vectorielle Qdrant ; DataHub commun utilise ClickHouse pour la traçabilité et l’évaluation des données. Le document nomme explicitement le rôle avancé du service commun. Ces technologies apportent une recherche sémantique et de l’analytique, sans présenter un modèle ML entraîné comme réalisé.

Dimensionnement

+ +Le projet comporte six capacités métier individuelles et un service commun DataHub. Les services sont intégrés progressivement : contrats et données, appels synchrones, événements, sécurité puis déploiement. Les choix de moteurs distincts impliquent des configurations et des stratégies de reprise propres à chaque service. + +## Ce qui fonctionne au stade S4

Les dix processus Spring Boot démarrent ; Config Server, Eureka et Gateway sont réels ; les routes /design exposent l’état de conception. Chaque service possède ses propres modèles et configuration de base prévue. Le portail React affiche les statuts réels des /design. Aucun catalogue vectoriel, métier météo, notification, broker ni base hétérogène n’est prétendu opérationnel en mode local. Le profil secure est codé ; la stack Docker nécessite une vérification ultérieure car Docker n’est pas installé sur la machine de préparation.

Données échanges et sécurité

REST / OpenFeign

Frontend > Gateway > Activity. Activity appelle Weather, Forecast, Air Quality et User via OpenFeign et découverte Eureka. Alert consulte ces mêmes contrats au besoin. Données : cityCode, plage temporelle, subject autorisé, contraintes, valeurs météo/AQI avec unités et provenance. Timeout connexion 2s, lecture 4s, aucun retry implicite sur commande non idempotente. Budgets agrégés et parallélisation à valider au développement. JWT transmis uniquement vers les services autorisés et identité /me contrôlée ; ce relais métier est prévu, non implémenté dans les stubs.

Qdrant est consulté uniquement par Activity : embedding du texte utilisateur, filtre catalogue et recherche top-k ; les contraintes météo restent numériques. Le modèle d'embedding et sa dimension sont versionnés. DataHub n'appelle aucune base métier.

RabbitMQ

Exchange durable topic meteo.events. Une file par consommateur : alert.events, datahub.events. Routage par type/version, schéma d'événement commun, publisher confirms. Outbox transactionnelle pour SQL ; mécanisme propre Mongo/Redis/Qdrant à concevoir et valider (transaction Mongo en replica set, stream Redis, tâche d'ingestion reprise). Ne pas prétendre à une transaction atomique générique entre des moteurs hétérogènes et RabbitMQ.

Message : eventId, eventType, schemaVersion=1, producer, occurredAt UTC, aggregateId, correlationId, payload ; pas de mot de passe ni données personnelles inutiles. Les consommateurs acquittent après traitement durable, retry exponentiel borné (3), puis file de quarantaine. Rejeu manuel tracé. Aucun « exactly once » annoncé : livraison au moins une fois et effets idempotents.

Matrice

Producteur · Appelants REST · Événements · Consommateurs
Weather · Activity, Alert · WeatherUpdated · Alert, DataHub
Forecast · Activity, Alert · ForecastUpdated · Alert, DataHub
Air Quality · Activity, Alert · AirQualityUpdated · Alert, DataHub
User · Activity, Alert · PreferencesChanged, ConsentWithdrawn · Alert, DataHub (payload minimisé)
Activity · Frontend via Gateway · RecommendationComputed · DataHub
Alert · Frontend via Gateway · AlertRaised, AlertAcknowledged · DataHub
DataHub · Administrateur via Gateway · aucun obligatoire · –

Pannes

503 si dépendance indispensable absente ; données périmées signalées avec date et source ; pas de score inventé. Gateway renvoie 503 s'il n'existe pas d'instance disponible. Config Server requis au démarrage ; instance déjà démarrée conserve sa configuration chargée. Eureka est un annuaire, pas le chemin des messages métier. Keycloak indisponible empêche une nouvelle connexion ; un JWT existant peut être validé localement avec des clés en cache dans sa période de validité. Pas de désactivation automatique de sécurité. Si DataHub est absent, les files conservent les événements jusqu'à la limite de rétention monitorée.

Sécurité prévue : OIDC code avec PKCE S256 ; JWT avec signature, émetteur, audience et expiration vérifiés ; autorisations métier /me et consentement. Le profil local est explicitement anonyme et lié à 127.0.0.1. Le profil secure existe dans Gateway et services ; l'intégration complète Keycloak/Frontend reste à réaliser après validation.

Weather – Nour Maroufi

Responsabilité et frontière métier

Acquérir et historiser les conditions météo courantes.. Propriété exclusive : weather_db ; moteur **PostgreSQL**. Aucun accès direct d'un autre service à cette base.

Données et justification

Snapshots horodatés, ville, coordonnées, température, vent, humidité, fournisseur, fetchedAt, qualityFlags.

PostgreSQL : contraintes, index ville/date et transactions pour snapshots et outbox. Aucune lecture interbase.

Modèle logique : WeatherSnapshot(id UUID PK, cityCode, fetchedAt, source); WeatherReading(snapshotId FK, temperatureC, windKmh, humidityPct).

API prévue

GET /api/weather/{city} > 200,400,404,503 ; GET /api/weather/{city}/history > 200,400.

Ces URI décrivent la cible à développer après validation. Seuls /api/weather/design et Actuator sont réellement servis en S4. Les autres chemins du service renvoient 501. Réponses d'erreur structurées, validation de l'identité et des entrées à implémenter avec le métier.

Appelants, appelés et échanges

Produit WeatherUpdated ; appelé par Activity et Alert ; aucune dépendance métier synchrone sortante.

Synchrone OpenFeign : réponse immédiate nécessaire à la recommandation ou à la personnalisation. Charge utile : ville, dates, unités, provenance et horodatage ; préférences minimales. Asynchrone RabbitMQ : découpler ingestion, classification et analyse, sans bloquer une lecture utilisateur. Chaque événement suit event-envelope-v1.schema.json.

Panne : API fournisseur : timeout 2s/4s ; cache au plus 30 minutes signalé avec sa date ; 503 si aucune donnée exploitable.

Projet et configuration

Projet Java 17/Spring Boot indépendant dans Backend/weather-service. Modèles de conception Java, interfaces Feign le cas échéant, configuration centralisée et tests du contrat de squelette. Le profil local démarre sans bases externes ; aucune connexion métier n'est revendiquée. Le profil databases prépare le moteur prévu ; Docker reste à vérifier sur une machine équipée.

Projet initialisé : Backend/weather-service. Il compile et démarre. Modèles et contrats disponibles dans le dossier du service.

Forecast – Dhia Borzli

Responsabilité et frontière métier

Servir les prévisions horaires et préserver leurs versions.. Propriété exclusive : forecast_db ; moteur **MongoDB**. Aucun accès direct d'un autre service à cette base.

Données et justification

Runs immuables de prévision, heures imbriquées, issuedAt si disponible, fetchedAt, variable, horizon, provenance.

MongoDB : document ForecastRun avec tableau hours ; index unique ville/source/version, TTL seulement sur caches, jamais sur références évaluées.

Modèle logique : ForecastRun(_id, cityCode, issuedAt?, fetchedAt, source, hours:[{validAt,temperatureC,windKmh,rainMm,rainProbabilityPct}]).

API prévue

GET /api/forecasts/{city} > 200,400,404,503 ; GET /api/forecasts/{city}/runs > 200,400.

Ces URI décrivent la cible à développer après validation. Seuls /api/forecasts/design et Actuator sont réellement servis en S4. Les autres chemins du service renvoient 501. Réponses d'erreur structurées, validation de l'identité et des entrées à implémenter avec le métier.

Appelants, appelés et échanges

Produit ForecastUpdated ; appelé par Activity et Alert ; DataHub reçoit les versions.

Synchrone OpenFeign : réponse immédiate nécessaire à la recommandation ou à la personnalisation. Charge utile : ville, dates, unités, provenance et horodatage ; préférences minimales. Asynchrone RabbitMQ : découpler ingestion, classification et analyse, sans bloquer une lecture utilisateur. Chaque événement suit event-envelope-v1.schema.json.

Panne : Timeout fournisseur, cache étiqueté ; 503 si heures manquantes indispensables ; aucune interpolation silencieuse.

Projet et configuration

Projet Java 17/Spring Boot indépendant dans Backend/forecast-service. Modèles de conception Java, interfaces Feign le cas échéant, configuration centralisée et tests du contrat de squelette. Le profil local démarre sans bases externes ; aucune connexion métier n'est revendiquée. Le profil databases prépare le moteur prévu ; Docker reste à vérifier sur une machine équipée.

Projet initialisé : Backend/forecast-service. Il compile et démarre. Modèles et contrats disponibles dans le dossier du service.

Alert – Mohamed Hedi Bejaoui

Responsabilité et frontière métier

Classifier les situations et gérer le cycle de vie des alertes.. Propriété exclusive : alert_db ; moteur **Redis**. Aucun accès direct d'un autre service à cette base.

Données et justification

Règles versionnées, abonnements consentis, alertes actives, clé de déduplication et état d'acquittement.

Redis : ensembles et hashes pour état actif/TTL et contrôle atomique de doublons. AOF activé, rétention explicite ; archive durable des changements par événement vers DataHub. Limite : sans archive acquittée, ne pas purger.

Modèle logique : rule:{id} HASH ; subscription:{subject}:{id} HASH ; alert:{id} HASH ; dedup:{eventId}:{ruleVersion} STRING SET NX ; user-alerts:{subject} ZSET.

API prévue

GET /api/alerts/me > 200,401 ; POST /api/alerts/subscriptions > 201,400,401,409 ; PATCH /api/alerts/{id}/ack > 200,401,403,404.

Ces URI décrivent la cible à développer après validation. Seuls /api/alerts/design et Actuator sont réellement servis en S4. Les autres chemins du service renvoient 501. Réponses d'erreur structurées, validation de l'identité et des entrées à implémenter avec le métier.

Appelants, appelés et échanges

Consomme WeatherUpdated / ForecastUpdated / AirQualityUpdated ; appelle Weather, Forecast, Air Quality, User via OpenFeign ; produit AlertRaised / AlertAcknowledged.

Synchrone OpenFeign : réponse immédiate nécessaire à la recommandation ou à la personnalisation. Charge utile : ville, dates, unités, provenance et horodatage ; préférences minimales. Asynchrone RabbitMQ : découpler ingestion, classification et analyse, sans bloquer une lecture utilisateur. Chaque événement suit event-envelope-v1.schema.json.

Panne : Retry borné, file de quarantaine ; aucune nouvelle classe rassurante si données manquantes ; conserver état inconnu/stale et informer le client.

Projet et configuration

Projet Java 17/Spring Boot indépendant dans Backend/alert-service. Modèles de conception Java, interfaces Feign le cas échéant, configuration centralisée et tests du contrat de squelette. Le profil local démarre sans bases externes ; aucune connexion métier n'est revendiquée. Le profil databases prépare le moteur prévu ; Docker reste à vérifier sur une machine équipée.

Projet initialisé : Backend/alert-service. Il compile et démarre. Modèles et contrats disponibles dans le dossier du service.

Air Quality – Kaouther Nalouti

Responsabilité et frontière métier

Fournir la pollution prévue et son indice documenté.. Propriété exclusive : air_quality_db ; moteur **MySQL**. Aucun accès direct d'un autre service à cette base.

Données et justification

Mesures modélisées AQI avec échelle documentée, PM2.5, PM10, ozone, date de validité et provenance CAMS/Open-Meteo.

MySQL : séries structurées, contraintes et index ville/validAt. AQI européen conservé comme indice fournisseur ; ne pas le présenter comme classe officielle du projet.

Modèle logique : AirQualitySnapshot(id UUID PK, cityCode, validAt, fetchedAt, source); PollutantReading(snapshotId FK, europeanAqi, pm25, pm10, ozone).

API prévue

GET /api/air-quality/{city} > 200,400,404,503 ; GET /api/air-quality/{city}/history > 200,400.

Ces URI décrivent la cible à développer après validation. Seuls /api/air-quality/design et Actuator sont réellement servis en S4. Les autres chemins du service renvoient 501. Réponses d'erreur structurées, validation de l'identité et des entrées à implémenter avec le métier.

Appelants, appelés et échanges

Produit AirQualityUpdated ; appelé par Activity et Alert ; DataHub consomme les événements.

Synchrone OpenFeign : réponse immédiate nécessaire à la recommandation ou à la personnalisation. Charge utile : ville, dates, unités, provenance et horodatage ; préférences minimales. Asynchrone RabbitMQ : découpler ingestion, classification et analyse, sans bloquer une lecture utilisateur. Chaque événement suit event-envelope-v1.schema.json.

Panne : 503 si fournisseur indisponible et pas de cache frais ; indice absent distinct de zéro ; timeouts et contrôle de cohérence des unités.

Projet et configuration

Projet Java 17/Spring Boot indépendant dans Backend/air-quality-service. Modèles de conception Java, interfaces Feign le cas échéant, configuration centralisée et tests du contrat de squelette. Le profil local démarre sans bases externes ; aucune connexion métier n'est revendiquée. Le profil databases prépare le moteur prévu ; Docker reste à vérifier sur une machine équipée.

Projet initialisé : Backend/air-quality-service. Il compile et démarre. Modèles et contrats disponibles dans le dossier du service.

User – Ahmed Ben Mahmoud

Responsabilité et frontière métier

Gérer profils métier, villes favorites et consentements.. Propriété exclusive : user_db ; moteur **MariaDB**. Aucun accès direct d'un autre service à cette base.

Données et justification

Profil applicatif indexé par subject Keycloak, préférences, favoris, consentement versionné et date de retrait. Aucun mot de passe.

MariaDB : relations profil/préférences/favoris, transactions et contraintes d'unicité. Même famille SQL que MySQL mais instance et credentials distincts.

Modèle logique : UserProfile(subject VARCHAR PK, displayName, consentVersion, consentAt); Preference(subject FK, activityType, maxWind, maxAqi); FavoriteCity(subject FK, cityCode).

API prévue

GET /api/users/me > 200,401,404 ; PUT /api/users/me/preferences > 200,400,401 ; DELETE /api/users/me/consent > 204,401.

Ces URI décrivent la cible à développer après validation. Seuls /api/users/design et Actuator sont réellement servis en S4. Les autres chemins du service renvoient 501. Réponses d'erreur structurées, validation de l'identité et des entrées à implémenter avec le métier.

Appelants, appelés et échanges

Appelé par Activity et Alert avec identité ou rôle de service contrôlé ; produit PreferencesChanged / ConsentWithdrawn.

Synchrone OpenFeign : réponse immédiate nécessaire à la recommandation ou à la personnalisation. Charge utile : ville, dates, unités, provenance et horodatage ; préférences minimales. Asynchrone RabbitMQ : découpler ingestion, classification et analyse, sans bloquer une lecture utilisateur. Chaque événement suit event-envelope-v1.schema.json.

Panne : Profil indisponible : pas de personnalisation silencieuse. 503 ; accès /me lié au sub du JWT. Retrait de consentement prioritaire et propagé.

Projet et configuration

Projet Java 17/Spring Boot indépendant dans Backend/user-service. Modèles de conception Java, interfaces Feign le cas échéant, configuration centralisée et tests du contrat de squelette. Le profil local démarre sans bases externes ; aucune connexion métier n'est revendiquée. Le profil databases prépare le moteur prévu ; Docker reste à vérifier sur une machine équipée.

Projet initialisé : Backend/user-service. Il compile et démarre. Modèles et contrats disponibles dans le dossier du service.

Activity – Mohamed Aziz Sayhi

Responsabilité et frontière métier

Classer les créneaux selon météo, air et préférences.. Propriété exclusive : activity_db ; moteur **Qdrant (vectoriel)**. Aucun accès direct d'un autre service à cette base.

Données et justification

Catalogue d'activités et embeddings sémantiques avec modèle/dimension/version ; payload explicatif ; candidats et contexte météo utilisés à la requête.

Qdrant : recherche vectorielle filtrée sur catalogue d'activités. Le vecteur compare préférences/descriptions ; les contraintes météo/AQI sont contrôlées ensuite sur les données numériques. Aucun score vectoriel présenté comme une probabilité de sécurité.

Modèle logique : Collection activity_catalog : point UUID, vector[384] normalisé avec modèle paraphrase-multilingual-MiniLM-L12-v2 version figée, payload {activityCode,title,description,embeddingVersion,minAge?,tags}. Dimension/modèle à vérifier avant ingestion, toute migration utilise une nouvelle collection. Les résultats météo ne sont pas stockés comme vérité vectorielle.

API prévue

POST /api/activities/search > 200,400,401,503 ; POST /api/activities/recommendations > 200,400,401,422,503.

Ces URI décrivent la cible à développer après validation. Seuls /api/activities/design et Actuator sont réellement servis en S4. Les autres chemins du service renvoient 501. Réponses d'erreur structurées, validation de l'identité et des entrées à implémenter avec le métier.

Appelants, appelés et échanges

Appelle Weather, Forecast, Air Quality et User via OpenFeign ; recherche Qdrant via client HTTP interne ; produit RecommendationComputed (identité pseudonymisée). Appelé par frontend via Gateway.

Synchrone OpenFeign : réponse immédiate nécessaire à la recommandation ou à la personnalisation. Charge utile : ville, dates, unités, provenance et horodatage ; préférences minimales. Asynchrone RabbitMQ : découpler ingestion, classification et analyse, sans bloquer une lecture utilisateur. Chaque événement suit event-envelope-v1.schema.json.

Panne : Dépendance indispensable indisponible : 503 sans classement. Qdrant indisponible : recherche catalogue indisponible, aucune similarité inventée. Une activité choisie explicitement pourra être évaluée sans recherche si validé au contrat.

Projet et configuration

Projet Java 17/Spring Boot indépendant dans Backend/activity-service. Modèles de conception Java, interfaces Feign le cas échéant, configuration centralisée et tests du contrat de squelette. Le profil local démarre sans bases externes ; aucune connexion métier n'est revendiquée. Le profil databases prépare le moteur prévu ; Docker reste à vérifier sur une machine équipée.

Projet initialisé : Backend/activity-service. Il compile et démarre. Modèles et contrats disponibles dans le dossier du service.

DataHub – Toute l’équipe

Responsabilité et frontière métier

Historiser les événements et produire des rapports de qualité et d’évaluation.. Propriété exclusive : datahub_db ; moteur **ClickHouse**. Aucun accès direct d’un autre service à cette base.

Données et justification

Projections analytiques d’événements, provenance, contrôles de qualité, versions de prévision et références temporelles. Pas de données de profil directement identifiantes.

ClickHouse : stockage en colonnes et agrégats par ville/date/horizon sur événements append-only. Déduplication logique par eventId via ReplacingMergeTree et lecture FINAL/argMax pour les rapports ; pas de garantie d’unicité OLTP.

Modèle logique : EventLog(eventId UUID, eventType, occurredAt, source, aggregateId, schemaVersion, payload, receivedAt) ENGINE ReplacingMergeTree(receivedAt) ORDER BY eventId ; ForecastEvaluation(runId, validAt, horizonHours, variable, referenceKind, absError).

API prévue

GET /api/datahub/quality > 200,400,401 ; GET /api/datahub/forecast-errors > 200,400,401,422 ; GET /api/datahub/events > 200,400,401.

Ces URI décrivent la cible à développer après validation. Seuls /api/datahub/design et Actuator sont réellement servis en S4. Les autres chemins du service renvoient 501. Réponses d’erreur structurées, validation de l’identité et des entrées à implémenter avec le métier.

Appelants, appelés et échanges

Commun à toute l’équipe ; consomme WeatherUpdated / ForecastUpdated / AirQualityUpdated / AlertRaised / PreferencesChanged / RecommendationComputed ; aucun accès direct aux autres bases.

Synchrone OpenFeign : réponse immédiate nécessaire à la recommandation ou à la personnalisation. Charge utile : ville, dates, unités, provenance et horodatage ; préférences minimales. Asynchrone RabbitMQ : découpler ingestion, classification et analyse, sans bloquer une lecture utilisateur. Chaque événement suit event-envelope-v1.schema.json.

Panne : Acquittement RabbitMQ après écriture réussie ; redélivrance et déduplication logique, retry borné et quarantaine ; aucun rapport « complet » si des événements manquent.

Projet et configuration

Projet Java 17/Spring Boot indépendant dans Backend/datahub-service. Modèles de conception Java, interfaces Feign le cas échéant, configuration centralisée et tests du contrat de squelette. Le profil local démarre sans bases externes ; aucune connexion métier n’est revendiquée. Le profil databases prépare le moteur prévu ; Docker reste à vérifier sur une machine équipée.

Projet initialisé : Backend/datahub-service. Il compile et démarre. Modèles et contrats disponibles dans le dossier du service.

Organisation Git planning et risques

Branche d'intégration main, branches feature/weather, feature/forecast, feature/alert, feature/air-quality, feature/user, feature/activity, feature/datahub. Une PR par changement, revue par un autre étudiant, Maven verify obligatoire en CI.

Chaque microservice dispose d'une branche de développement et de tickets dédiés. Les changements sont intégrés par pull request, avec revue et vérification des contrats.

Outil de gestion retenu : **GitHub Issues + jalons + labels par service**. Un ticket contient responsable attendu, critères de recette et statut via labels todo, in-progress, review, done. Un tableau Kanban local est fourni dans TASKS.md.

Planning

S4 : revue des frontières, modèles, APIs, nombre de services, bases et soutenance de conception. Après validation : S5 squelettes/bases et ingestion ; S6 OpenFeign et contrats ; S7 RabbitMQ et reprise ; S8 sécurité et consentement ; S9 Docker et recette ; S10 soutenance finale. Planning proposé, à adapter au calendrier enseignant.

Risques

Sept moteurs augmentent l'exploitation : images et configurations figées, un profil de développement par service et intégration progressive. Modèle vectoriel non évalué : jeu de requêtes de pertinence relu par l'équipe avant l'intégration. API externe : cache daté et limites de débit. Données manquantes : réponse explicite. Authentification : JWT validé côté Gateway et service. Intégration tardive : stabiliser les contrats et effectuer des revues régulières.

État vérifié du dépôt

Dépôt public : https://github.com/sayhi124/MeteoSentinel-AWD . Sept tickets sont ouverts, un par service, avec labels et jalon Validation 1. La CI Maven et React a réussi sur GitHub.